15. Machine Learning Model Selection

15.1 Framing the Problem

Predicting `risk_category` is a three-class classification over 48 tabular features, with classes distributed 35 / 45 / 20. The data is entirely tabular: no images, no sequences, no text. There are 22,000 records, which is enough to fit a moderately complex model and not enough to justify one that must learn its own representations.

The class imbalance is mild but material. A classifier that always answered *Medium* would score 45% accuracy, so accuracy alone cannot demonstrate that a model has learned anything. Weighted F1 was adopted as the selection criterion, and both are reported.

15.2 Candidates

Five classifiers were trained and compared, chosen to span the approaches that are actually competitive on tabular data rather than to enumerate algorithms for their own sake.

Random Forest — bagged decision trees with feature subsampling at each split. Robust to feature scale, tolerant of collinearity, and cheap to explain with `TreeExplainer`.

XGBoost, **LightGBM** and **CatBoost** — three gradient-boosting implementations. Boosting fits trees sequentially against the residual of the ensemble so far, which typically extracts more signal than bagging when signal exists to be extracted.

Neural network — a two-hidden-layer perceptron, 128 and 64 units, with early stopping. Included as a control rather than a serious contender: on tabular data of this size, gradient-boosted trees generally outperform dense networks, and confirming that on this dataset is more useful than assuming it.

A logistic regression baseline was not fitted, which is an omission. It would have established whether the non-linear models were earning their complexity, and its absence is noted in Section 26.

15.3 Protocol

The processed table was split 80/20 with stratification on the label under `random_state=42`, giving 17,600 training and 4,400 test records. Every model saw the same partition.

All five were first fitted with library-default hyperparameters and scored on the held-out set. Selection was made on weighted F1 against those defaults, so that the comparison was between algorithms rather than between tuning budgets. The Random Forest won at F1 = 0.5949, ahead of LightGBM at 0.5830 and CatBoost at 0.5827.

The winner alone was then tuned by randomised search over 20 candidate configurations with three-fold cross-validation, which raised its weighted F1 to 0.6006. The selected configuration:

Hyperparameter	Value
n_estimators	200
max_depth	10
min_samples_split	5
min_samples_leaf	1
max_features	sqrt
criterion	gini
bootstrap	True

Five-fold stratified cross-validation on the tuned model gives weighted F1 = **0.5996 ± 0.0032**, confirming that the held-out figure is not an artefact of one fortunate partition.

Feature scaling was fitted during preprocessing and saved, but is not applied. Tree ensembles are invariant to monotone rescaling of individual features, so standardisation would change nothing for the selected model. It would have mattered for the neural network, and its absence there is a confound: the network was evaluated on unscaled inputs, which disadvantages it. Since the network was included as a control and lost by a wide margin, this does not affect the selection, but the reported figure understates what a properly scaled network would achieve.

15.4 Comparison

Model	Accuracy	Precision (w)	Recall (w)	F1 (w)	ROC AUC (ovr, w)
Random Forest (tuned)	**0.6039**	0.6177	0.6039	**0.6006**	**0.7461**
LightGBM	0.5845	0.5923	0.5845	0.5830	0.7359
CatBoost	0.5841	0.5906	0.5841	0.5827	0.7290
XGBoost	0.5620	0.5686	0.5620	0.5609	0.7164
Neural Network	0.5332	0.5447	0.5332	0.5000	0.6550

One caveat attaches to this table, and it should not be buried. The Random Forest row reports the **tuned** model, while the other four report **untuned defaults**, because `train.py` overwrites the winner's entry with its post-tuning metrics. The table therefore compares a tuned model against four that were not tuned, and overstates the Random Forest's margin by roughly half a point of F1. The selection decision is unaffected — it was taken before tuning, on the untuned figures, where the Random Forest also led — but a reader comparing rows is not comparing like with like. The untuned Random Forest scored F1 = 0.5949.

15.5 Why the Random Forest Won

Gradient boosting outperforming bagging is the ordinary expectation on tabular problems, and here it did not. The explanation lies in the structure of the label rather than in any property of the implementations.

As Section 12.4 sets out, `risk_category` is a threshold on a linear score to which Gaussian noise of standard deviation 8.0 has been added, against a signal whose standard deviation is 7.64. Roughly half the variance in the underlying score is irreducible. Boosting's advantage is its capacity to fit residual structure across successive rounds; when the residual is predominantly noise, that capacity becomes a liability, and the boosted models overfit it. Bagging averages independently grown trees and cannot chase a residual it never computes.

Section 23 makes this concrete by deriving the Bayes-optimal accuracy for this label — the ceiling that no classifier can exceed — and showing that the Random Forest reaches 98.9% of it. There was very little residual structure left for boosting to find.

15.6 The Secondary Model

A second Random Forest was trained to predict `investment_preference` across five classes, using the same features and an identical protocol but no hyperparameter search.

It achieves accuracy 0.2586, weighted F1 0.1892 and ROC AUC 0.4949. The majority-class baseline is 0.2869 and chance-level ROC AUC is 0.5000. **The model performs worse than always answering "Mutual Funds", and its ranking of classes is indistinguishable from random.**

This is not a tuning failure and could not be repaired by a better algorithm. Section 22.3 traces it to the construction of the label.

> **Figure 15.1** — *Model comparison bar chart,* five models on the x-axis, grouped bars for accuracy / weighted F1 / ROC AUC. Annotate the Random Forest bar to indicate it is the tuned model, per the caveat in Section 15.4. Place in Section 15.4.
>
> **Figure 15.2** — *ROC curves, one-vs-rest,* three curves for Low / Medium / High from the tuned Random Forest, with the macro-average and the diagonal. Generate from `predict_proba` on the test split. Place in Section 15.4.
>
> **Table 15.1** — *Randomised search space,* listing the distribution sampled for each hyperparameter alongside the selected value. Source from `train.py` . Place in Section 15.3.